

RELATÓRIO DE ESTRUTURA DE DADOS I: SIMULADOR DE ESTACIONAMENTO COM MANOBRISTA

Autores: Pedro Lucas Cortez e Rubens de Carvalho Veloso

Instituição: Instituto Federal do Piauí (IFPI) - Análise e Desenvolvimento de Sistemas

1. Introdução

Este relatório apresenta a implementação do simulador de "Estacionamento com Manobrista", desenvolvido em Linguagem C como avaliação final da disciplina de Estrutura de Dados I. O objetivo principal é demonstrar o gerenciamento dinâmico de memória, manipulação de ponteiros e a aplicação prática de estruturas lineares em um cenário restrito de movimentação de veículos.

2. Descrição do Sistema

O sistema consiste em uma aplicação de terminal interativa com menu que permite:

- Registrar a entrada de veículos informando placa e nome do proprietário.
- Alocar o veículo na fileira principal (Pilha) ou direcioná-lo para a fila de espera caso a capacidade limite seja atingida.
- Processar a retirada de um veículo específico, desempilhando e contabilizando as manobras dos veículos obstrutores e recolocando-os em seguida.
- Promover automaticamente o primeiro veículo da fila de espera para a vaga liberada.
- Realizar busca sequencial de veículos cadastrados.
- Exibir a ocupação atual e o histórico de saídas ordenado por manobras.

3. Estruturas de Dados Utilizadas

- **Pilha (LIFO):** A fileira principal do estacionamento foi modelada como Pilha, pois possui entrada e saída por uma única extremidade (topo). Quando um veículo no fundo precisa sair, os veículos à sua frente são desempilhados temporariamente.
- **Fila Encadeada (FIFO):** Representa a fila de espera na rua. Preserva rigorosamente a ordem de chegada (First In, First Out) para a ocupação das vagas que forem sendo liberadas.
- **Lista Encadeada Simples:** Mantém o histórico dos veículos que já deixaram o local, permitindo alocação dinâmica sem limitação prévia de tamanho.

4. Algoritmos de Busca e Ordenação

- **Busca Sequencial:** Percorre os elementos da pilha comparando a string da placa informada pelo usuário para verificar a localização do veículo.
- **Bubble Sort:** Algoritmo aplicado sobre a lista encadeada de histórico para ordenar os relatórios em ordem decrescente de acordo com o número de manobras sofridas.

5. Implementação e Gerenciamento de Memória

A solução foi dividida em funções modulares. A alocação dinâmica é tratada via `malloc` na criação dos nós de Fila e Lista encadeada. Ao encerrar a aplicação, a função `liberar_memoria` percorre as estruturas executando `free()` em todos os ponteiros remanescentes para evitar vazamento de memória (*memory leak*).

6. Testes e Resultados

O programa foi testado no ambiente Linux (WSL/Ubuntu) com a ferramenta `gcc`. Foram homologados os seguintes cenários:

1. Inserção de veículos até o limite da capacidade da pilha.
2. Redirecionamento correto de novos veículos para a fila de espera.
3. Retirada de veículo do fundo com desempilhamento e incremento das manobras dos carros da frente.
4. Promoção automática do primeiro da fila de espera para o estacionamento após a liberação da vaga.
5. Liberação total de memória no encerramento.

7. Conclusão

O projeto cumpriu todos os requisitos propostos de maneira funcional e enxuta. A integração entre Pilha, Fila e Lista resolveu com precisão o problema de simulação do estacionamento.

8. Contribuição dos Integrantes

- **Pedro Lucas Cortez:** Implementação da lógica de Pilha/Fila, manobras e documentação do repositório.
- **Rubens de Carvalho Veloso:** Implementação da Lista Encadeada de histórico, ordenação Bubble Sort e testes de alocação de memória.